BÁO CÁO THỰC HÀNH LAB02

Viết Device Driver và Quản lý File System

Môn học: LOS201 – Embedded Linux Systems

Bản thảo đối chiếu với đề LAB-02 và ảnh kết quả thực hành ngày 29/09–01/10/2026. Những mục chưa có bằng chứng được đánh dấu để bổ sung trước khi nộp.

Name: Trần Thiên Phúc

ID Student: SE203798

Lớp: IC2008

Ngày thực hiện: 25/09/2026

1. Thông tin sinh viên và môi trường

Sinh viên: Trần Thiên Phúc; MSSV: SE203798; lớp: IC2008; ngày thực hiện ghi trên bìa: 25/09/2026. Nhóm: [BỔ SUNG NẾU CÓ]. Ảnh minh chứng được chụp trong các ngày 29/09–01/10/2026.

Ubuntu Host: thư mục làm việc /home/phuctran/LOS201/lab02; kernel Host 7.0.0-34-generic được nhìn thấy trong đường dẫn modinfo -n nandsim. Hệ thống đích: ARMv7 trên QEMU vexpress-a9, kernel 5.15.0 theo vermagic của module. Toolchain: arm-linux-gnueabihf-; [BỔ SUNG OUTPUT arm-linux-gnueabihf-gcc --version ĐỂ XÁC NHẬN PHIÊN BẢN TRONG LẦN NỘP NÀY].

Mở đầu

LAB02 nối tiếp kết quả LAB01: kernel ARM và initramfs BusyBox khởi động được trên QEMU. Bài này triển khai character device driver lab2_driver để trao đổi dữ liệu qua /dev/lab2, cung cấp trạng thái qua /proc và /sys, thử nghiệm JFFS2 trên NAND giả lập bằng nandsim ở Ubuntu Host, rồi nạp module tự động qua BusyBox init trên máy ARM ảo. Các phần A, B, D là cùng một nhánh driver và rootfs của hệ thống ARM; phần C được thực hiện riêng trên kernel Ubuntu Host. Vì vậy các đường dẫn /dev và /proc giống tên nhưng thuộc hai kernel khác nhau tùy ngữ cảnh.

Kết quả đã thấy: module ARM tải được, dữ liệu ghi qua /dev/lab2 đọc lại được, /proc và ba thuộc tính /sys đổi theo dữ liệu, JFFS2 đọc và giữ file sau khi tháo/lắp lại, BusyBox rcS tự tải module khi QEMU boot. Báo cáo phân biệt bằng chứng hiện có với những log đầy đủ mà đề yêu cầu nhưng ảnh chưa thể xác nhận.

2. Sơ đồ kiến trúc driver

User space	Lời gọi hệ thống	VFS	Driver	Bộ đệm trong kernel
Shell / test_driver.sh truy cập /dev/lab2	open(), write(), read(), close()	Dựa vào major 240, minor 0; chuyển lời gọi qua character device	file_operations: lab2_open, lab2_write, lab2_read, lab2_release	device_buffer[1024], buffer_len; mutex bảo vệ dữ liệu

Luồng dữ liệu: userspace → system call → VFS → file_operations → device_buffer; chiều đọc trả dữ liệu theo chiều ngược lại bằng copy_to_user(). register_chrdev(240, "lab2", &lab2_fops)

đăng ký phép ánh xạ major sang driver; /dev/lab2 là điểm truy cập có major 240, minor 0. Đây là sơ đồ kiến trúc theo mục 5.3 của đề; các ô mô tả đúng luồng đã thử trong QEMU.

```
/sys/class/lab2_class/lab2/
  ├── dev              (240:0)
  ├── buffer_len       (chỉ đọc)
  ├── open_count       (chỉ đọc)
  └── last_data        (chỉ đọc)
```

/proc/lab2_info là một bản tóm tắt dạng văn bản về driver, major, kích thước bộ đệm, độ dài dữ liệu, số lần mở và dữ liệu cuối. /sys/class/lab2_class/lab2/ biểu diễn thiết bị trong mô hình kobject và tách từng thuộc tính thành file. Những file ảo này không phải bản sao dữ liệu được lưu trên flash; chúng phản ánh trạng thái kernel hiện thời. devtmpfs trong /dev cung cấp node để mở driver; trong thực hành, rcS kiểm tra và tạo /dev/lab2 khi cần.

3. Phần A – Viết Kernel Module Character Device

A.1. Cấu trúc Kernel Module cơ bản

Mục tiêu. Xây dựng module có init/exit và bốn thao tác open, read, write, release; giao tiếp từ shell ARM qua /dev/lab2.

Tại Ubuntu Host, mã nguồn lab2_driver.c được triển khai với MODULE_LICENSE("GPL"), DEVICE_NAME="lab2", CLASS_NAME="lab2_class", BUFFER_SIZE=1024 và DRIVER_MAJOR=240. Hàm lab2_init đăng ký major bằng register_chrdev, tạo class và device; lab2_exit thu hồi entry procfs/sysfs (sau khi bổ sung Phần B), device/class và major. file_operations gắn các lời gọi VFS với hàm driver. open tăng open_count; release ghi thông báo đóng. write giới hạn số byte để chừa byte kết thúc chuỗi và dùng copy_from_user; read tôn trọng *offset/EOF và dùng copy_to_user. DEFINE_MUTEX bảo vệ buffer cùng các giá trị thống kê liên quan trong bản code được quan sát.

```
static struct file_operations lab2_fops = {
    .owner = THIS_MODULE,
    .open = lab2_open, .read = lab2_read,
    .write = lab2_write, .release = lab2_release,
};
/* Ý chính: register_chrdev(240, "lab2", &lab2_fops); */
```

Khối trên nêu các ánh xạ cần giải thích, không thay thế toàn bộ mã nguồn nộp riêng. Việc gọi copy_from_user/copy_to_user là điều kiện để truy cập bộ nhớ userspace một cách có kiểm tra; memcpy trực tiếp tới con trỏ __user có thể gây lỗi truy cập và thiếu kiểm tra ranh giới.

A.2. Viết Makefile cho Kernel Module

Makefile dùng KBuild của đúng kernel ARM đã build: obj-m += lab2_driver.o, ARCH=arm, CROSS_COMPILE=arm-linux-gnueabihf-. KBuild cần cây kernel tương thích với hệ thống đích để tạo module có ABI và vermagic phù hợp. Đường dẫn thực tế trong buổi làm là /home/phuctran/LOS201/lab02/kernel-work/linux-5.15; khác đường dẫn ví dụ ~/embedded_lab1/... trong đề, nhưng cùng nguyên tắc dùng kernel nguồn đã chuẩn bị.

```
make -C /home/phuctran/LOS201/lab02/kernel-work/linux-5.15 \
+    M=/home/phuctran/LOS201/lab02/driver \
+    ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- modules
```

A.3. Build và kiểm tra Module

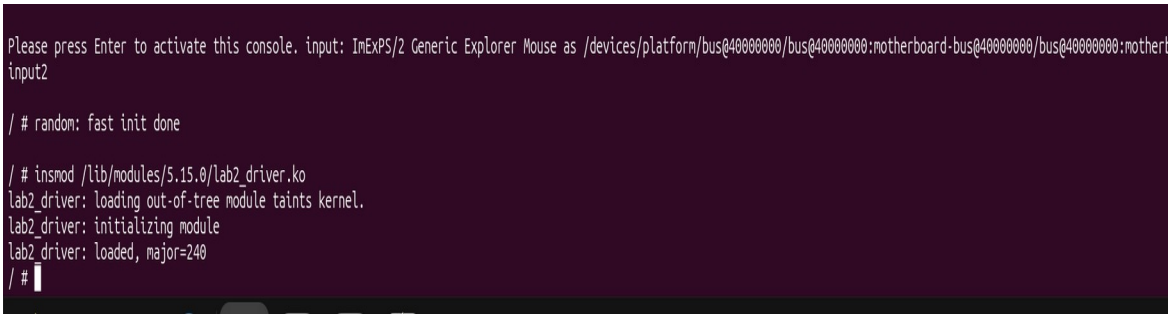
Lần build đầu có lỗi do tùy chọn ISA ARM của cây kernel chưa tương thích với bộ biên dịch chéo. Sau khi điều chỉnh cờ build, KBuild tạo lab2_driver.ko. Lệnh file xác nhận ELF 32-bit ARM; modinfo -F vermagic trả 5.15.0 SMP mod_unload ARMv7 p2v8. Module được chép vào rootfs/lib/modules/5.15.0/lab2_driver.ko rồi đóng gói cùng initramfs. Đường dẫn host và đường dẫn /lib/modules/... trong QEMU chỉ cùng một file tại hai thời điểm: trước và sau đóng gói rootfs.

```
file /home/phuctran/LOS201/lab02/driver/lab2_driver.ko
modinfo -F vermagic /home/phuctran/LOS201/lab02/driver/lab2_driver.ko
```



Hình 1. Ubuntu Host xác nhận module là ELF ARM 32-bit và vermagic 5.15.0 SMP mod_unload ARMv7 p2v8. (Ảnh chụp ngày 29/09/2026.)

Trong shell BusyBox của QEMU, insmod /lib/modules/5.15.0/lab2_driver.ko in ra “initializing module” và “loaded, major=240”; thông báo “out-of-tree module taints kernel” cho biết đây là module ngoài cây kernel, tự nó không phải lỗi load. Kết quả lsmod và node /dev/lab2 ở Phần D còn xác nhận module chạy sau boot tự động.

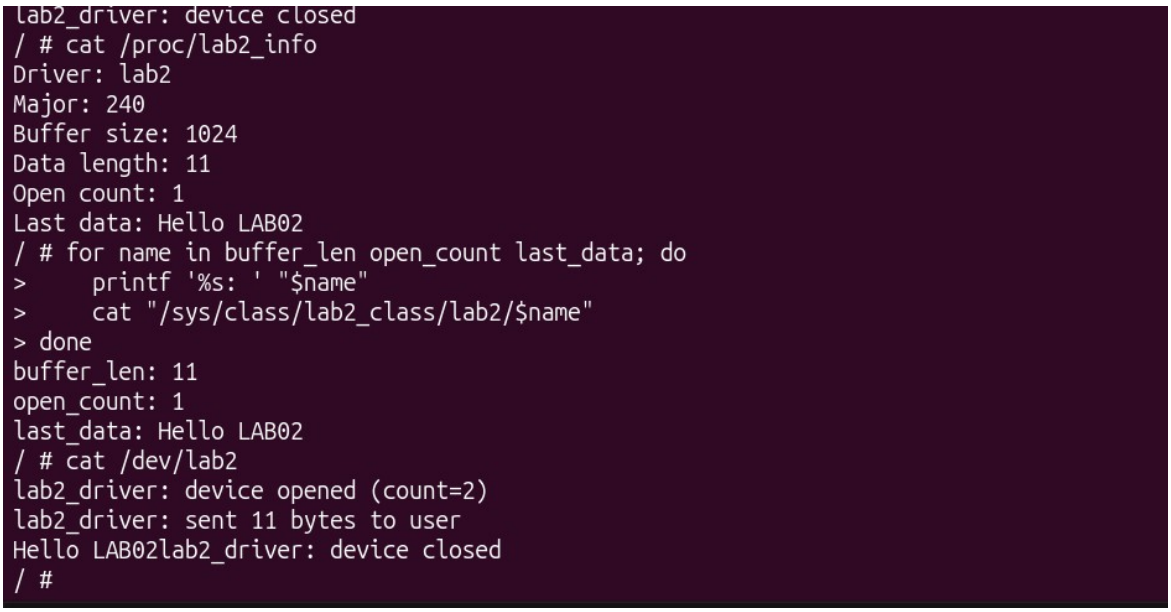


Hình 2. Shell ARM trong QEMU: insmod thành công và driver báo major 240. (Ảnh chụp ngày 29/09/2026.)

A.4. Tạo script test driver trong userspace

Script test_driver.sh được đặt tại /usr/bin trong rootfs, dùng lsmod để xác nhận module, bảo đảm node /dev/lab2 tồn tại, ghi một thông điệp, đọc lại, lặp với Message_1–Message_3 rồi xem dmesg. Trong ảnh thực hành ban đầu, thao tác thủ công printf 'Hello LAB02' > /dev/lab2 báo nhận 11 byte; cat /dev/lab2 trả Hello LAB02 và log báo gửi 11 byte. Điều này chứng minh nhánh write và read dùng cùng buffer. Khi cat mở file một lần nữa, open_count tăng lên 2; output từ log xen giữa ký tự do console kernel và stdout dùng chung màn hình.

```
printf 'Hello LAB02' > /dev/lab2
cat /dev/lab2
```



Hình 3. QEMU đọc lại Hello LAB02 từ /dev/lab2; log ghi số byte đã gửi và lần mở tiếp theo. (Ảnh chụp ngày 29/09/2026.)

[CHÈN ẢNH: toàn bộ output modinfo driver/lab2_driver.ko trên Ubuntu Host; dmesg | grep lab2 và output đầy đủ test_driver.sh trong QEMU ARM sau lần boot Phần D — hiện chưa có ảnh hoặc cần tớ gửi lại]

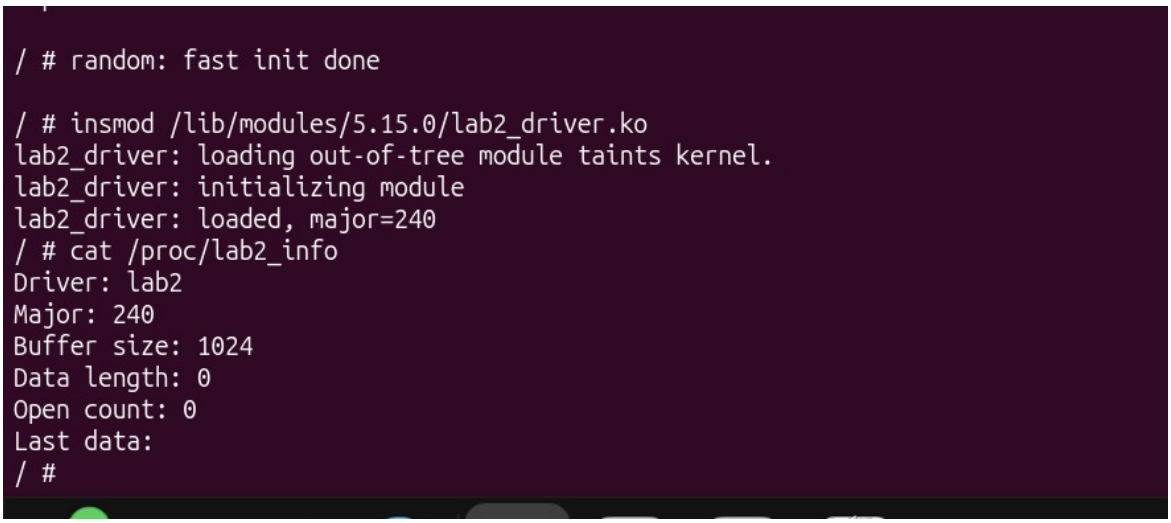
Nhận xét. Có bằng chứng build đúng ARM, nạp module và trao đổi dữ liệu. Ảnh chưa cho thấy nguyên văn output đầy đủ của modinfo, test_driver.sh hoặc lệnh lọc dmesg, nên các hạng mục log này còn thiếu dù thao tác chính đã hoạt động.

4. Phần B – Giao tiếp qua procfs và sysfs

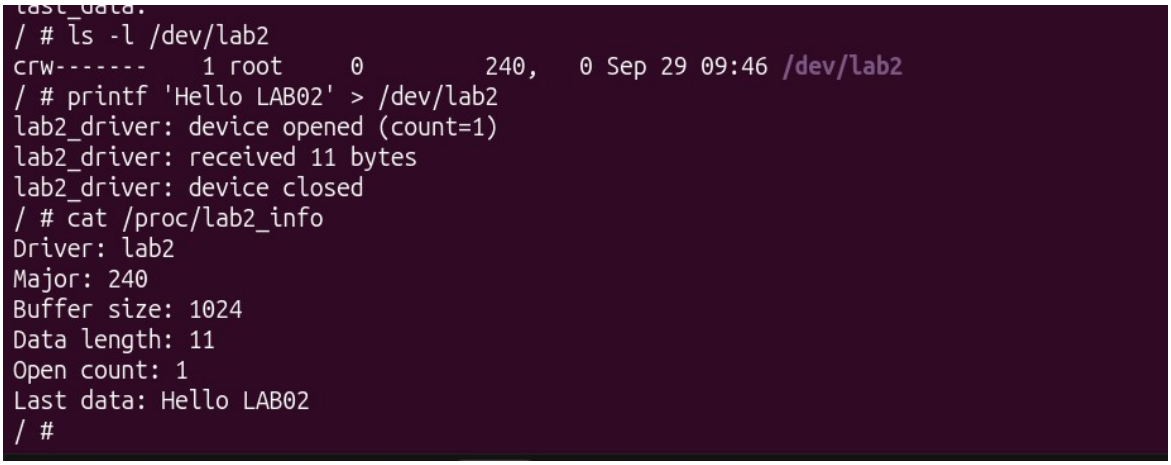
B.1. Thêm giao diện /proc vào driver

Mục tiêu. Quan sát trạng thái nội bộ mà không phải đọc lại thiết bị, đồng thời kiểm tra số liệu thay đổi sau khi write.

Driver tạo /proc/lab2_info với proc_create và proc_ops; lab2_proc_open dùng single_open, lab2_proc_show dùng seq_printf để xuất nhiều dòng, seq_read/seq_lseek/single_release quản lý vòng đời đọc. Kết quả trước khi ghi trong QEMU có Buffer size: 1024, Data length: 0, Open count: 0. Sau printf 'Hello LAB02' > /dev/lab2, ảnh cho thấy Data length: 11, Open count: 1, Last data: Hello LAB02. Vì /proc là giao diện tổng hợp, các con số được lấy từ trạng thái của driver chứ không phải file tồn tại trên đĩa.



Hình 4. QEMU: /proc/lab2_info trước khi write, độ dài dữ liệu và số lần mở bằng 0. (Ảnh chụp ngày 29/09/2026.)



Hình 5. QEMU: /proc/lab2_info sau khi write, 11 byte và lần mở đầu tiên được ghi nhận. (Ảnh chụp ngày 29/09/2026.)

B.2. Thêm giao diện sysfs vào driver

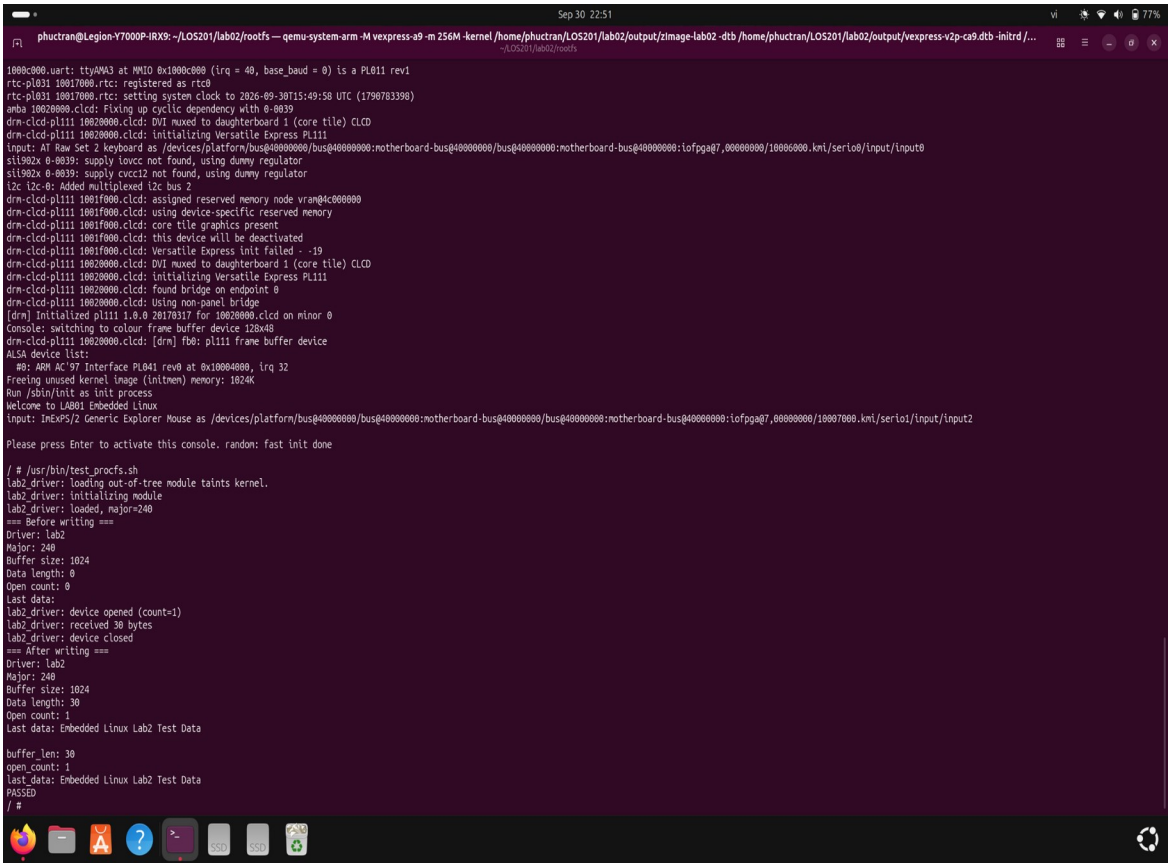
Driver khai báo DEVICE_ATTR_RO cho buffer_len, open_count, last_data và đăng ký nhóm thuộc tính trên kobject của lab2_device. Các file này chỉ đọc trong bản lab đã thực hiện; mục tiêu mở rộng “có thể đọc/ghi” của phần B là mô tả khả năng chung của sysfs, không có bằng chứng về attribute ghi được trong thiết kế cụ thể. Khi đọc ba file sau thao tác write, chúng lần lượt trả 11, 1, Hello LAB02, thống nhất với /proc. Thư mục /sys/class/lab2_class/lab2/ là symlink vào cây thiết bị ảo, trong khi /dev/lab2 là node thao tác I/O.

```
crw-------    1 root     0         240,   0 Sep 29 09:46 /dev/lab2
/ # printf 'Hello LAB02' > /dev/lab2
lab2_driver: device opened (count=1)
lab2_driver: received 11 bytes
lab2_driver: device closed
/ # cat /proc/lab2_info
Driver: lab2
Major: 240
Buffer size: 1024
Data length: 11
Open count: 1
Last data: Hello LAB02
/ # for name in buffer_len open_count last_data; do
>     printf '%s: ' "$name"
>     cat "/sys/class/lab2_class/lab2/$name"
> done
buffer_len: 11
open_count: 1
last_data: Hello LAB02
/ #
```

Hình 6. QEMU: ba attribute sysfs trả 11, 1 và Hello LAB02 sau khi ghi thiết bị. (Ảnh chụp ngày 29/09/2026.)

B.3. Script test_procfs.sh và kiểm tra

Script trong rootfs dùng set -e; nếu chưa có /proc/lab2_info thì nạp module, nếu /dev/lab2 chưa có thì lấy major/minor từ /sys/class/lab2_class/lab2/dev để mknod. Sau đó script đọc /proc trước, ghi chuỗi “Embedded Linux Lab2 Test Data” vào device, đọc /proc sau và đọc ba file sysfs. Lần thử riêng ở QEMU ngày 30/09 hiển thị Data length từ 0 thành 30, Open count từ 0 thành 1, Last data đúng chuỗi trên; ba thuộc tính sysfs là 30, 1, cùng chuỗi; cuối màn hình là PASSED. Điều này xác thực tính nhất quán của hai giao diện với buffer driver.



Hình 7. QEMU ở lần thử Phần B: test_procfs.sh hiển thị /proc trước/sau write, toàn bộ ba thuộc tính /sys và PASSED; trích vùng terminal của ảnh gốc. (Ảnh chụp ngày 30/09/2026.)

Nhận xét. /proc thích hợp với bản thống kê nhiều dòng để đọc tổng quan trong bài lab; /sys thích hợp với các thuộc tính thiết bị đơn lẻ có cấu trúc ổn định. Script B chạy thành công ở lần boot trước khi tích hợp auto-load của Phần D; không thể dùng ảnh này để khẳng định đã chạy lại script sau lần boot D.

5. Phần C – MTD Subsystem và JFFS2 File System

C.1. Tổng quan về MTD và JFFS2

Mục tiêu. Dùng NAND giả lập để xóa, ghi image JFFS2, mount và xác nhận dữ liệu còn sau umount/mount.

Toàn bộ C.2–C.5 chạy trên Ubuntu Host, với sudo cho thao tác MTD; QEMU ARM không tham gia phần thử flash này. MTD cung cấp giao diện raw flash với eraseblock, page, OOB và bad block, khác block device dùng cho ext4. NAND đã xóa ở trạng thái 0xFF, không ghi đè tùy ý từ 0 lên 1; JFFS2 ghi theo kiểu log, gom dữ liệu còn hiệu lực rồi xóa block để tái sử dụng, đồng thời quản lý wear leveling và nén.

C.2. Kích hoạt nandsim và kiểm tra MTD trên Host

Trước nandsim, /proc/mtd của Host có mtd0 tên BIOS. Do đó ví dụ /dev/mtd0 trong đề tuyệt đối không áp dụng nguyên xi cho máy này. Sau khi nạp nandsim, mtd1 mang tên “NAND simulator partition 0”; mọi lệnh ghi/xóa thực tế được ràng buộc với mtd1 bằng kiểm tra /sys/class/mtd/mtd1/name. mtdinfo /dev/mtd1 báo 64,0 MiB, 512 eraseblock, mỗi block 131072 byte (128 KiB), page 2048 byte và OOB 64 byte. Đây là thông số dùng để chọn -e 0x20000, -s 0x800 cho mkfs.jffs2.

```
sudo modprobe nandsim first_id_byte=0x20 second_id_byte=0xa2 third_id_byte=0x00
fourth_id_byte=0x15
cat /proc/mtd
mtdinfo /dev/mtd1
```

```
Setting up libiniparser4:amd64 (4.2.6-1build1)…
Setting up mtd-utils (1:2.3.0-1ubuntu2)…
Processing triggers for man-db (2.13.1-1build1)…
Processing triggers for libc-bin (2.43-2ubuntu2.4)…
phuctran@Legion-Y7000P-IRX9:~/LOS201/lab02/rootfs$ mtdinfo /dev/mtd1
mtd1
Name:                           NAND simulator partition 0
Type:                           nand
Eraseblock size:                131072 bytes, 128.0 KiB
Amount of eraseblocks:          512 (67108864 bytes, 64.0 MiB)
Minimum input/output unit size: 2048 bytes
Sub-page size:                  512 bytes
OOB size:                       64 bytes
Character device major/minor:   90:2
Bad blocks are allowed:         true
Device is writable:             true

phuctran@Legion-Y7000P-IRX9:~/LOS201/lab02/rootfs$
```

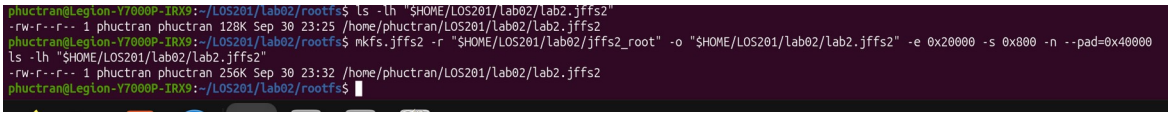
Hình 8. Ubuntu Host: mtdinfo xác nhận mtd1 là NAND simulator 64 MiB, eraseblock 128 KiB, page 2048 byte. (Ảnh chụp ngày 30/09/2026.)

C.3. Xóa và Format JFFS2

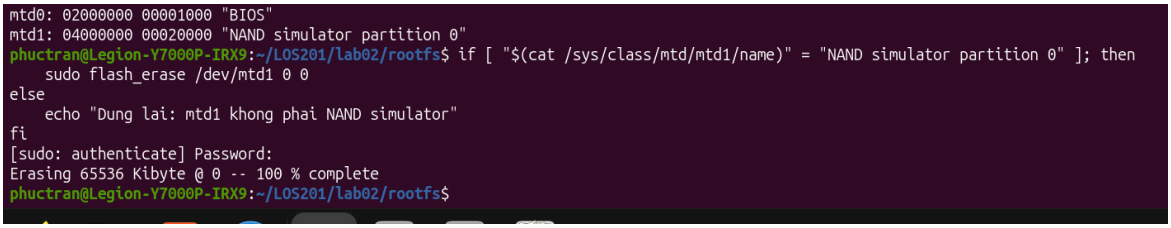
Host tạo jffs2_root/hello.txt chứa “LAB02 JFFS2 test”. Image đầu tiên dùng tham số eraseblock chưa phù hợp; sau khi đối chiếu mtdinfo, tạo lại /home/phuctran/LOS201/lab02/lab2.jffs2 với eraseblock 0x20000, page 0x800, --pad=0x40000 và -n; file cuối có kích thước 256 KiB. Trước

khi ghi, script kiểm tra đúng tên mtd1, rồi flash_erase /dev/mtd1 0 0 hoàn tất 100% cho 64 MiB. nandwrite -p ghi image vào block 0 và 1 tại offset 0x0 và 0x20000. Dòng cảnh báo “Written 127 blocks containing only 0xff bytes” đã xuất hiện; kiểm chứng mount/read ở bước sau mới cho thấy image được kernel nhận.

```
mkfs.jffs2 -r "$HOME/LOS201/lab02/jffs2_root" \
+  -o "$HOME/LOS201/lab02/lab2.jffs2" -e 0x20000 -s 0x800 -n --pad=0x40000
# Chỉ chạy với mtd1 sau khi xác minh tên là NAND simulator partition 0
sudo flash_erase /dev/mtd1 0 0
sudo nandwrite -p /dev/mtd1 "$HOME/LOS201/lab02/lab2.jffs2"
```



Hình 9. Ubuntu Host: tạo lại lab2.jffs2 đúng eraseblock 0x20000, page 0x800; image cuối 256 KiB. (Ảnh chụp ngày 30/09/2026.)



Hình 10. Ubuntu Host: kiểm tra đúng mtd1 rồi flash_erase báo xóa 64 MiB thành công. (Ảnh chụp ngày 30/09/2026.)

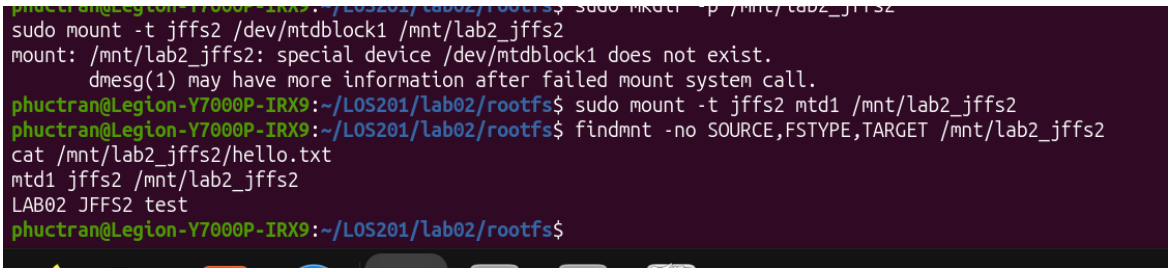


Hình 11. Ubuntu Host: nandwrite ghi JFFS2 image lên hai eraseblock đầu của NAND giả lập. (Ảnh chụp ngày 30/09/2026.)

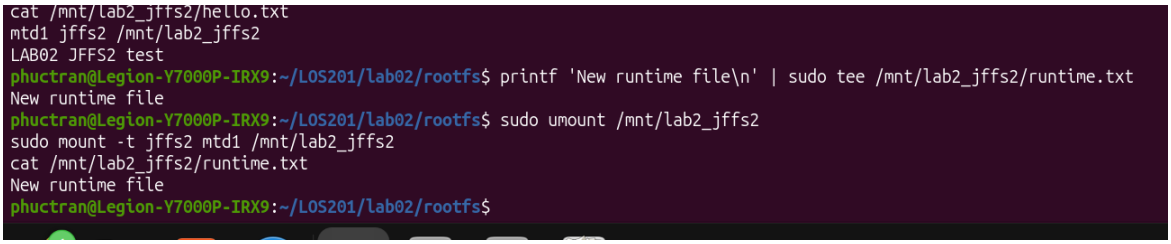
C.4. Mount và thao tác với JFFS2

Lệnh mount -t jffs2 /dev/mtdblock1 thất bại vì node /dev/mtdblock1 không tồn tại trong môi trường Host này. Sau đó mount -t jffs2 mtd1 /mnt/lab2_jffs2 thành công; findmnt xác nhận SOURCE=mtd1, FSTYPE=jffs2 và cat hello.txt trả “LAB02 JFFS2 test”. Host ghi thêm runtime.txt với nội dung “New runtime file”, umount rồi mount lại mtd1; cat runtime.txt vẫn trả chính chuỗi đó. Kết luận chỉ giới hạn ở tính bền vững qua một chu kỳ tháo/lắp trong NAND giả lập đang chạy, chưa phải kiểm chứng sau tắt máy hoặc mất điện.

```
sudo mount -t jffs2 mtd1 /mnt/lab2_jffs2
findmnt -no SOURCE,FSTYPE,TARGET /mnt/lab2_jffs2
cat /mnt/lab2_jffs2/hello.txt
printf 'New runtime file\n' | sudo tee /mnt/lab2_jffs2/runtime.txt
sudo umount /mnt/lab2_jffs2
sudo mount -t jffs2 mtd1 /mnt/lab2_jffs2
cat /mnt/lab2_jffs2/runtime.txt
```



Hình 12. Ubuntu Host: mount qua tên MTD mtd1 thành công sau khi node mtdblock1 không có; hello.txt đọc được. (Ảnh chụp ngày 30/09/2026.)



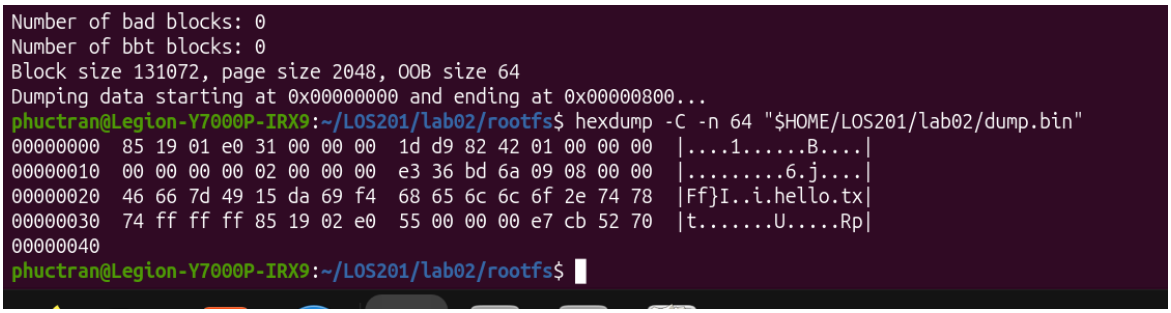
Hình 13. Ubuntu Host: runtime.txt vẫn chứa New runtime file sau khi umount và mount lại. (Ảnh chụp ngày 30/09/2026.)

Đề còn yêu cầu đọc info.txt, logs/boot.log và ghi nối tiếp boot.log; ảnh hiện có chỉ chứng minh hello.txt và runtime.txt. Không coi những file mẫu của đề là kết quả đã thu được trên máy này.

[CHÈN ẢNH: cat /mnt/lab2_jffs2/info.txt, cat /mnt/lab2_jffs2/logs/boot.log và log sau khi ghi nối tiếp, nếu các file này đã được tạo trong lần thực hành — hiện chưa có ảnh hoặc cần tớ gửi lại]

C.5. Kiểm tra và dump MTD raw data

Sau khi umount, nanddump đọc vùng đầu từ /dev/mtd1 vào dump.bin. hexdump -C -n 64 hiển thị dãy 85 19 ở offset 0, dạng little endian của magic JFFS2 0x1985, và tên hello.txt trong dữ liệu thô. Đây là bằng chứng độc lập rằng image đã được ghi lên MTD. Sau đó rmmod nandsim; /proc/mtd chỉ còn BIOS mtd0, cho thấy thiết bị giả lập đã được gỡ.



Hình 14. Ubuntu Host: hexdump của NAND giả lập có magic JFFS2 85 19 và tên hello.txt. (Ảnh chụp ngày 30/09/2026.)

Nhận xét. Thứ tự xác nhận thiết bị → đọc geometry → tạo image đúng eraseblock → xóa → ghi → mount → kiểm tra persistence → dump là cần thiết. Lỗi node mtdblock và việc mtd0 là BIOS đã được xử lý bằng kiểm tra tên partition và mount bằng mtd1.

6. Phần D – Tích hợp BusyBox Init và Auto-load Module

D.1. Cập nhật init script để auto-load module

Mục tiêu. Đưa module đã build vào chuỗi khởi động BusyBox để /dev/lab2 sẵn sàng sau khi vào shell ARM.

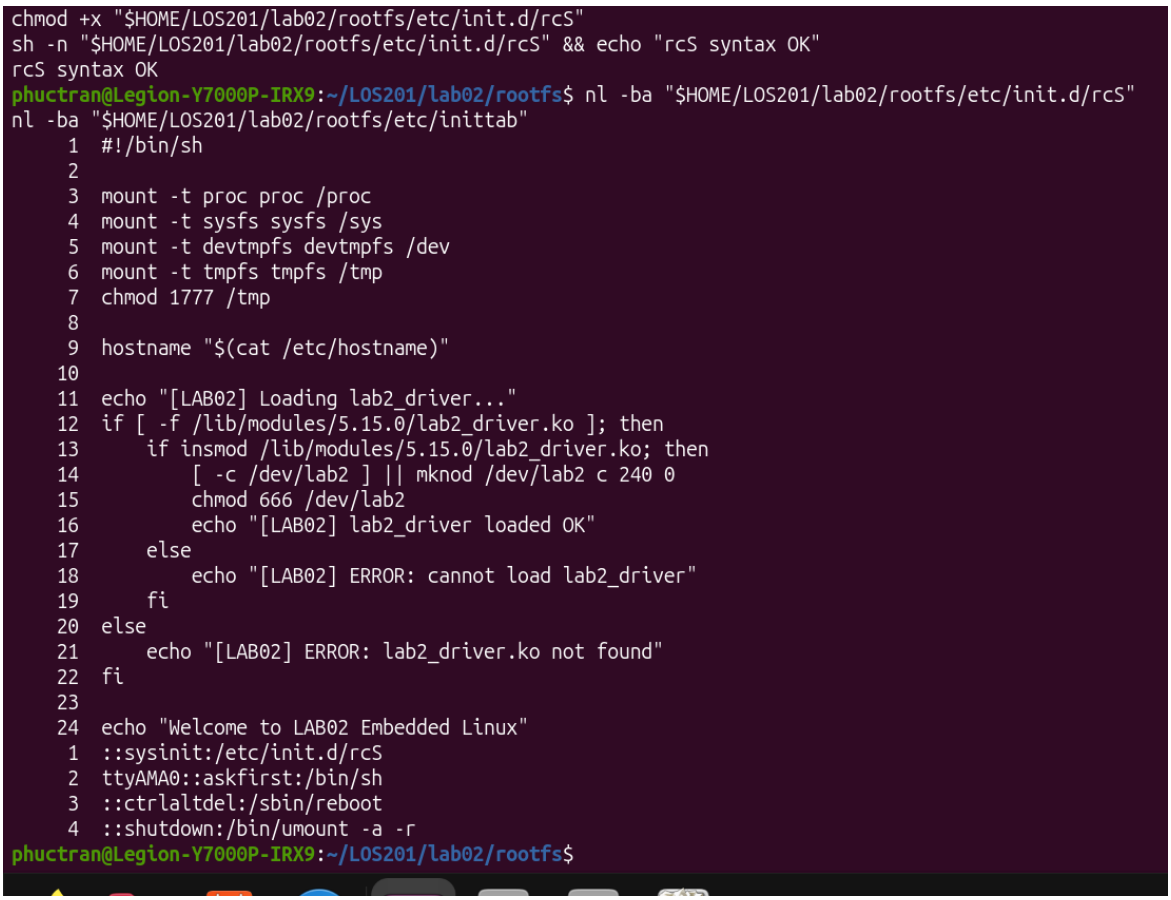
Ở Ubuntu Host, rootfs/etc/init.d/rcS được sửa để mount proc, sysfs, devtmpfs và tmpfs, đặt hostname, kiểm tra file /lib/modules/5.15.0/lab2_driver.ko, gọi insmod, bảo đảm /dev/lab2 là character node major 240 minor 0 rồi chmod 666. Các mount pseudo filesystem cần có trước để /proc, /sys và /dev phản ánh kernel ARM đang chạy. Script kiểm tra exit status của insmod và in thông báo thành công hoặc lỗi. Ảnh kiểm tra cú pháp sh -n cho thấy rcS syntax OK.

```
mount -t proc proc /proc
mount -t sysfs sysfs /sys
mount -t devtmpfs devtmpfs /dev
mount -t tmpfs tmpfs /tmp
if [ -f /lib/modules/5.15.0/lab2_driver.ko ]; then
    if insmod /lib/modules/5.15.0/lab2_driver.ko; then
        [ -c /dev/lab2 ] || mknod /dev/lab2 c 240 0
        chmod 666 /dev/lab2
        echo "[LAB02] lab2_driver loaded OK"
    else echo "[LAB02] ERROR: cannot load lab2_driver"; fi
fi
```

Đoạn trên tóm lược logic đã quan sát trong rcS, không khẳng định từng dòng là bản sao nguyên văn của file nộp. Trong bản chụp, rcS tiếp tục chạy tới welcome ngay cả khi nhánh insmod lỗi; cách xử lý lỗi nghiêm trọng được bàn ở Câu 5.

D.2. Cập nhật /etc/inittab

BusyBox init là tiến trình đầu tiên của userspace: đọc /etc/inittab, chạy /etc/init.d/rcS ở action sysinit, rồi mở shell askfirst trên ttyAMA0. Bản thực hành còn có ctrlaltdel và shutdown. Nếu rcS không được chỉ định hoặc không executable, module không thể tự load theo cách này. Ảnh kiểm tra hiển thị cả rcS và inittab sau khi chỉnh sửa.



Hình 15. Ubuntu Host: nội dung rcS với insmod, mknod và inittab với action sysinit/askfirst. (Ảnh chụp ngày 01/10/2026.)

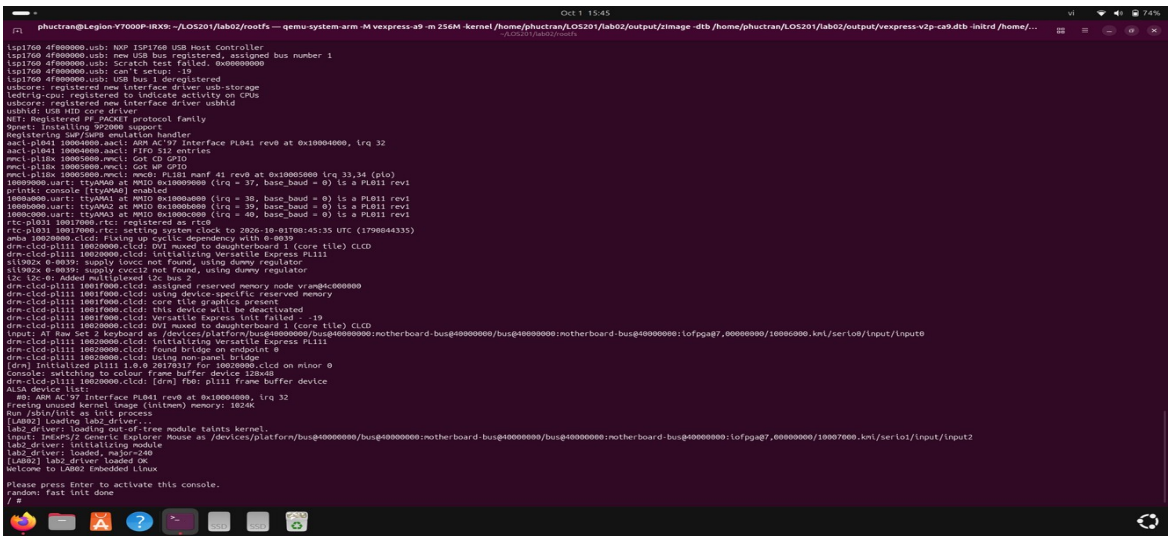
D.3. Kiểm tra cấu trúc rootfs trước khi đóng gói

Host xác nhận module nằm tại rootfs/lib/modules/5.15.0/lab2_driver.ko, hai script test ở rootfs/usr/bin, rcS ở rootfs/etc/init.d và inittab ở rootfs/etc. Việc sao chép module vào rootfs là bắt buộc: QEMU chỉ thấy nội dung initramfs, không tự thấy file trong thư mục driver của Ubuntu Host. Kết quả file/vermagic của A.3 giúp tránh đóng gói nhầm module dành cho Host.

D.4. Đóng gói initramfs và boot trên QEMU

Host dùng find . | cpio -H newc -o | gzip để tạo output/rootfs-lab02-D.cpio.gz (ảnh báo khoảng 1,1 MiB và 3482 blocks). Khi chạy qemu-system-arm -M vexpress-a9 -m 256M với zImage, vexpress-v2p-ca9.dtb và initramfs D, kernel ARM vào /sbin/init, rcS in “[LAB02] Loading lab2_driver...”, driver báo “loaded, major=240”, rcS báo “lab2_driver loaded OK” và shell / # xuất hiện. Các dòng này chứng minh thứ tự boot → init → rcS → insmod → shell. Lệnh QEMU thực tế dùng 256M, không phải dòng ví dụ 512M trong đề.

```
cd /home/phuctran/LOS201/lab02/rootfs
find . -print0 | cpio --null -o -H newc | gzip -9 > \
  /home/phuctran/LOS201/lab02/output/rootfs-lab02-D.cpio.gz
qemu-system-arm -M vexpress-a9 -m 256M \
  -kernel /home/phuctran/LOS201/lab02/output/zImage \
  -dtb /home/phuctran/LOS201/lab02/output/vexpress-v2p-ca9.dtb \
  -initrd /home/phuctran/LOS201/lab02/output/rootfs-lab02-D.cpio.gz \
  -append "console=ttyAMA0 rdinit=/sbin/init" -nographic
```

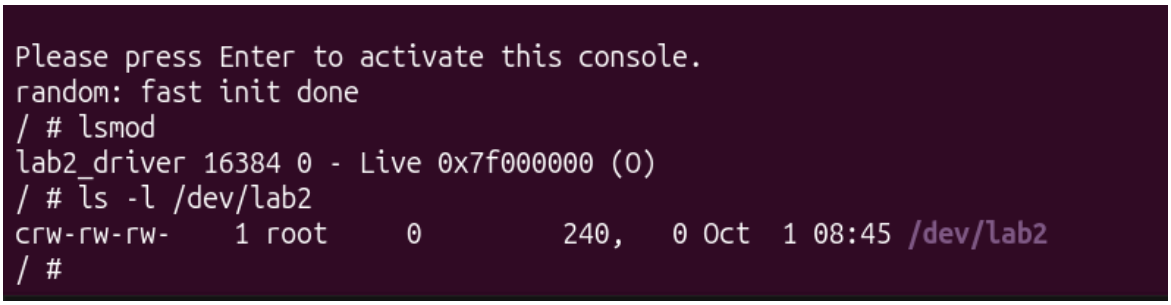


Hình 16. QEMU boot initramfs D; trích vùng cuối ảnh gốc có dòng rcS load module OK và dấu nhắc / #. (Ảnh chụp ngày 01/10/2026.)

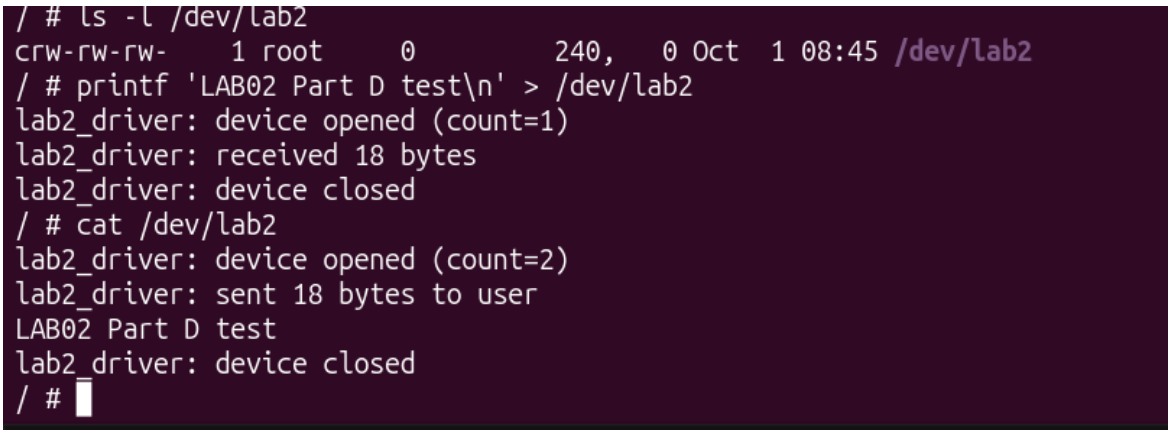
Vì ảnh boot gốc chứa rất nhiều dòng kernel, phần log quyết định được chép lại ở đây để dễ đọc: “[LAB02] Loading lab2_driver...”; “lab2_driver: initializing module”; “lab2_driver: loaded, major=240”; “[LAB02] lab2_driver loaded OK”; “Welcome to LAB02 Embedded Linux”; “/ #”. Đây là phần trích đọc được từ ảnh, không phải log toàn bộ quá trình boot.

D.5. Chạy kiểm tra toàn diện trong QEMU

Sau lần boot D, lsmod liệt kê lab2_driver và ls -l /dev/lab2 hiển thị character node 240, 0. Lệnh printf 'LAB02 Part D test\n' > /dev/lab2 làm driver báo nhận 18 byte; cat /dev/lab2 trả đúng “LAB02 Part D test” và driver báo gửi 18 byte. Đó là bằng chứng auto-load tạo môi trường đủ để I/O thủ công hoạt động. Tuy vậy, trong ảnh sau boot D chưa thấy đầu ra toàn bộ test_driver.sh, test_procfs.sh, đọc trực tiếp từng thuộc tính /proc và /sys, và dmesg | grep lab2 như D.5 yêu cầu. Ảnh B.3 là lần boot trước, không thay cho D.5.



Hình 17. QEMU sau auto-load: lsmod có lab2_driver và /dev/lab2 mang major 240 minor 0. (Ảnh chụp ngày 01/10/2026.)



Hình 18. QEMU sau auto-load: ghi và đọc lại LAB02 Part D test, driver báo nhận và gửi 18 byte. (Ảnh chụp ngày 01/10/2026.)

[CHÈN ẢNH: trong lần boot initramfs D, toàn bộ output test_driver.sh và test_procfs.sh — hiện chưa có ảnh hoặc cần tớ gửi lại]

[CHÈN ẢNH: trong lần boot initramfs D, cat /proc/lab2_info; ls và cat ba thuộc tính /sys/class/lab2_class/lab2/; dmesg | grep lab2 — hiện chưa có ảnh hoặc cần tớ gửi lại]

Nhận xét. Boot, auto-load, node thiết bị và I/O đã được xác nhận trực tiếp. D.5 mới có bằng chứng cho kiểm tra 1 và một thử đọc/ghi thủ công; log bắt buộc của kiểm tra 2–4 cần bổ sung trước khi đánh dấu hoàn tất hồ sơ.

7. Kiểm tra kết quả và so sánh filesystem

Bảng sau đối chiếu các filesystem mà đề yêu cầu. “Thời gian boot” là nhận xét định tính theo cách mount và quy mô dữ liệu, không phải số đo benchmark của buổi thực hành. Chỉ JFFS2 trên nandsim đã được thử trực tiếp trong phần C.

Tiêu chí	ext4	JFFS2	cramfs
Khả năng ghi	Đọc/ghi; có journaling.	Đọc/ghi trên raw flash.	Chỉ đọc.
Wear leveling	Không xử lý raw NAND; cần lớp flash/FTL bên dưới.	Có cơ chế quản lý hao mòn và block hỏng.	Không ghi khi chạy, không có wear leveling cho dữ liệu động.
Nén	Không nén tích hợp trong trường hợp dùng thông thường.	Có nén dữ liệu.	Nén image tĩnh.
Thời gian mount/boot	Nhanh trên block device phù hợp; tùy kích thước và kiểm tra journal.	Có thể chậm dần với dung lượng lớn vì quét flash khi mount.	Thường nhanh cho rootfs nhỏ, tĩnh.
Thiết bị phù hợp	eMMC/SD/SSD hoặc block device có lớp quản lý flash.	Raw NAND/NOR dung lượng nhỏ, cần cập nhật dữ liệu.	Rootfs/firmware tĩnh chỉ đọc.
Minh chứng trong lab	Không thử.	Đã mount/ghi/umount/mount lại trên nandsim.	Không thử.

Với thiết bị IoT có raw NAND 32 MB và cần OTA, lựa chọn đề xuất trong phạm vi ba filesystem của bài là JFFS2 cho phần cần ghi và duy trì trạng thái; đặt firmware vào vùng riêng và thiết kế hai slot A/B để tải image mới, xác minh checksum/chữ ký, đổi slot boot sau khi ghi thành công và quay lại bản cũ khi lỗi. Nếu firmware hầu như tĩnh, có thể dùng cramfs cho image chỉ đọc và JFFS2 cho dữ liệu cấu hình, nhưng cramfs không tự giải quyết OTA hay tính nguyên tử. ext4 chỉ phù hợp khi thiết bị cung cấp block abstraction/FTL. Đây là khuyến nghị thiết kế, chưa được hiện thực hoặc đo trên thiết bị 32 MB trong buổi lab.

8. Trả lời câu hỏi phân tích của đề

Câu 1. copy_to_user và copy_from_user

Con trỏ buf của read/write là địa chỉ ảo thuộc tiến trình userspace, còn device_buffer là vùng nhớ kernel. Kernel không được giả định địa chỉ userspace luôn có thể truy cập như con trỏ kernel: trang có thể chưa hiện diện, không hợp lệ hoặc bị thay đổi; memcpy không thực hiện phép kiểm tra cần thiết cho biên này. copy_from_user(device_buffer, buf, len) đưa dữ liệu từ userspace vào kernel khi write; copy_to_user(buf, device_buffer + *offset, len) làm chiều ngược lại khi read. Hai hàm kiểm tra truy cập và trả về số byte chưa sao chép; code cần xử lý lỗi, chẳng hạn -EFAULT, cùng giới hạn độ dài. Nếu thay bằng memcpy, thao tác có thể gây fault trong kernel hoặc đọc/ghi địa chỉ sai. Minh họa thực tế: thông điệp Hello LAB02 đi qua write/read trên QEMU và trả đúng 11 byte; đó là quan sát về kết quả, không phải một phép thử ép lỗi địa chỉ.

Câu 2. Mutex và race condition

Hai tiến trình có thể đồng thời mở /dev/lab2 và gọi write. Nếu không khóa, tiến trình A có thể chép nửa chuỗi vào device_buffer, B chép chuỗi khác, rồi A gán buffer_len theo độ dài của A. Kết quả đọc có thể là chuỗi trộn, độ dài không khớp hoặc dữ liệu cuối không đúng; open_count cũng có nguy cơ mất cập nhật khi hai lần tăng giao nhau. DEFINE_MUTEX(lab2_mutex) và mutex_lock/mutex_unlock tạo vùng tới hạn cho buffer và thông tin liên quan, để một lần ghi hoàn tất trước lần ghi tiếp theo. Ảnh B cho thấy số liệu nhất quán sau một lần write, song buổi thực hành chưa chạy phép thử hai process cùng ghi; ví dụ race là phân tích kỹ thuật, không phải output đã quan sát.

Câu 3. procfs và sysfs

procfs ban đầu phục vụ trạng thái tiến trình và các tệp thông tin kernel dạng văn bản; /proc/lab2_info trong lab là bảng thống kê nhiều dòng, tiện quan sát. sysfs ánh xạ thiết bị, class và kobject thành cây file/thuộc tính, thường một giá trị cho một file với quyền truy cập rõ ràng. Vì sysfs có mô hình thiết bị và ABI attribute có cấu trúc, thông tin mới gắn với một device nên đặt ở /sys, tránh mở rộng tùy tiện /proc thành kho API thiết bị. Trong driver này, buffer_len, open_count và last_data phù hợp ở /sys/class/lab2_class/lab2/; bản tóm tắt dễ đọc ở /proc là giao diện bổ trợ theo yêu cầu học tập. Ảnh B chứng minh cùng một lần write tạo ra giá trị 30/1/chuỗi tương ứng ở cả hai nơi.

Câu 4. Eraseblock của NAND và log structured JFFS2

NAND chỉ lập trình bit theo một chiều sau khi erase, và erase tác động cả eraseblock lớn; muốn ghi lại vùng đã dùng phải tìm block mới hoặc xóa cả block, không thể ghi đè trực tiếp tùy ý lên từng sector như block device mà ext4 nhìn thấy. ext4 làm việc trên giao diện sector do thiết bị hoặc FTL cung cấp; nếu là flash, chính FTL bên dưới xử lý erase, remap và hao mòn. JFFS2 ghi node dữ liệu/metadata mới theo log, đánh dấu phiên bản cũ không còn hiệu lực, gom các node còn sống sang block khác rồi xóa block có thể thu hồi. Trong lab, mtdinfo cho eraseblock 128 KiB; flash_erase xóa mtd1 trước nandwrite; image được mount và ghi runtime.txt rồi đọc lại sau remount. Đây là minh họa thực tế cho các bước raw NAND và filesystem, không chứng minh định lượng wear leveling.

Câu 5. Lỗi insmod trong BusyBox rcS

BusyBox init gọi rcS theo action sysinit; bản rcS quan sát được kiểm tra file và exit status của insmod, in ERROR nếu nạp thất bại, rồi tiếp tục đến echo welcome và shell, vì không có lệnh exit hoặc vòng lặp dừng trong nhánh lỗi. Do vậy shell vẫn có thể xuất hiện nhưng /dev/lab2 không dùng được. Nếu device này là chức năng thiết yếu để boot, rcS phải phân loại nó là lỗi nghiêm trọng: ghi log, dừng tiến trình init theo chính sách hệ thống, gọi rescue shell hoặc reboot vào slot firmware dự phòng. Nếu không thiết yếu, in cảnh báo, giữ dịch vụ phụ thuộc không khởi chạy và tiếp tục boot. Script cải thiện minh họa dưới đây đề xuất chính sách không nghiêm trọng; chưa được áp dụng/chạy trong ảnh thực hành:

```
if [ ! -f /lib/modules/5.15.0/lab2_driver.ko ]; then
    echo "WARN: lab2_driver.ko missing" >&2
    lab2_ready=0
elif ! insmod /lib/modules/5.15.0/lab2_driver.ko; then
    echo "WARN: insmod failed; check dmesg" >&2
    lab2_ready=0
elif [ ! -c /dev/lab2 ] && ! mknod /dev/lab2 c 240 0; then
    echo "WARN: cannot create /dev/lab2" >&2
    lab2_ready=0
else
    chmod 666 /dev/lab2 || echo "WARN: chmod failed" >&2
    lab2_ready=1
fi
# Chỉ khởi động dịch vụ cần /dev/lab2 khi lab2_ready=1.
```

So với rcS hiện tại, đoạn đề xuất kiểm tra cả mknod/chmod, tránh thông báo “loaded OK” bị hiểu nhầm là mọi bước thiết lập device đều thành công. Nếu chọn chế độ nghiêm trọng, thay nhánh cảnh báo bằng quy trình cứu hộ hoặc chuyển slot phù hợp, chứ không chỉ exit 1 rồi mặc nhiên cho rằng máy sẽ tắt.

9. Nhận xét và khó khăn

Khó khăn quan sát được	Cách xử lý và bài học
KBuild ban đầu lỗi tùy chọn ISA “selected processor does not support … in ARM mode”.	Điều chỉnh cờ kiến trúc khi build module với đúng kernel ARM; xác minh file là ELF ARM và vermagic 5.15.0 trước khi chép vào rootfs.
Trên Host mtd0 có tên BIOS, không phải NAND giả lập.	Đọc /proc/mtd và tên /sys/class/mtd/mtd1/name; chỉ dùng /dev/mtd1 trong nhánh kiểm tra tên khớp. Không áp dụng số mtd trong ví dụ của đề một cách máy móc.
Node /dev/mtdblock1 không tồn tại nên mount theo block node thất bại.	Mount JFFS2 bằng tên MTD mtd1, kiểm tra findmnt và đọc hello.txt; sau đó thử umount/mount lại.
Image JFFS2 đầu tiên chưa khớp eraseblock thực tế.	Dựa trên mtdinfo 128 KiB/page 2048 để tạo lại image bằng -e 0x20000 -s 0x800; image cuối 256 KiB.

Hai ranh giới quan trọng nhất của bài là biên userspace/kernel trong driver và biên Host/QEMU trong thao tác hệ thống. MTD/JFFS2 ở Phần C không nằm trong initramfs ARM của Phần D; BusyBox chỉ khởi động driver và cung cấp shell/script trong QEMU. /proc và /sys là trạng thái do driver/kernel sinh ra, còn JFFS2 lưu file thực trên raw NAND giả lập. Cùng một driver có ba “cửa”: /dev cho I/O, /proc cho thống kê tổng hợp và /sys cho từng thuộc tính.

10. Đối chiếu yêu cầu LAB02 và hồ sơ hiện có

Yêu cầu theo đề	Nội dung/bằng chứng trong báo cáo	Trạng thái
5.3 mục 1: thông tin sinh viên, kernel, GCC	Mục 1: họ tên, MSSV, lớp, ngày đã điền; kernel đích 5.15.0; cần nhóm (nếu có) và output phiên bản GCC trong lần nộp này.	Còn thiếu minh chứng GCC/nhóm
5.3 mục 2: sơ đồ luồng driver, cây sysfs	Mục 2 có sơ đồ bảng năm lớp và ba attribute.	Đủ nội dung
A.1–A.3: mã, Makefile, ARM module, modinfo	Mục 3, Hình 1–2; file/vermagic và insmod có ảnh; chưa có toàn bộ modinfo.	Thiếu một log
A.4: test script, /dev, lsmod, dmesg	Mục 3, Hình 3 và D Hình 17–18; đọc/ghi, lsmod có ảnh; thiếu script đầy đủ và dmesg lọc.	Thiếu log
B.1–B.3: proc trước/sau, sysfs và script	Mục 4, Hình 4–7; test_procfs.sh PASSED ở lần boot B.	Đủ cho B
C.2–C.5: mtdinfo, erase, write, mount, persistence, dump	Mục 5, Hình 8–14; thiếu info.txt và logs/boot.log theo mẫu đề.	Đủ lõi, thiếu file mẫu
D.1–D.4: rcS, inittab, rootfs, auto-load QEMU	Mục 6, Hình 15–18; boot log ảnh và node/IO.	Đủ lõi
D.5: cả hai script, /proc, /sys, dmesg sau auto-boot	Chỉ có lsmod/node và I/O thủ công ở lần D; mục 6 để placeholder.	Thiếu bằng chứng bắt buộc
5.3 mục 4: so sánh ≥5 tiêu chí và 32MB NAND OTA	Mục 7 có sáu tiêu chí và đề xuất.	Đủ nội dung
5.3 mục 5: năm câu hỏi VI	Mục 8 trả lời đủ năm câu, phân biệt suy luận và thực nghiệm.	Đủ nội dung
5.3 mục 6: ít nhất hai khó khăn	Mục 9 ghi bốn trường hợp và cách khắc phục.	Đủ nội dung
5.1–5.2: PDF, archive, logs, comment source	Có bản PDF và DOCX; chưa xác nhận archive/log boot và các comment trong source nộp.	Cần kiểm tra trước nộp

Số hình trong bảng là nhóm tham chiếu theo nội dung; nếu Word tự dịch trang, dùng chú thích hình ngay cạnh ảnh làm chuẩn đối chiếu. Kết luận “đủ lõi” nghĩa là thao tác chính có ảnh, không thay thế các tệp/log bổ sung mà đề liệt kê.

11. Kết luận

LAB02 đã nối được driver character với rootfs ARM chạy trong QEMU: module ARM 5.15.0 tải được, /dev/lab2 trao đổi dữ liệu, /proc và /sys phản ánh trạng thái, và rcS tự nạp module lúc boot. Trên Ubuntu Host, nandsim tạo NAND 64 MiB ở mtd1; JFFS2 được tạo với geometry tương ứng, ghi, mount và giữ runtime.txt sau một lần umount/mount. Các bằng chứng này đủ để mô tả chuỗi thực hành A–D một cách trung thực. Trước khi nộp theo mục V của đề, cần hoàn thiện log D.5, thông tin sinh viên/toolchain, kiểm tra comment source và đóng gói hồ sơ đúng tên.

12. Ảnh và thông tin cần bổ sung

• Họ tên Trần Thiên Phúc, MSSV SE203798 và lớp IC2008 đã được điền; cần xác nhận nhóm (nếu có) và chụp kết quả arm-linux-gnueabihf-gcc --version cho mục 1.

• Trên Ubuntu Host: chụp toàn bộ modinfo driver/lab2_driver.ko. Trong QEMU ARM: chụp dmesg | grep lab2. Hai log này phục vụ Phần A; dmesg sau auto-boot còn phục vụ D.5.

• Sau khi boot initramfs D trong QEMU ARM: chụp toàn bộ test_driver.sh, test_procfs.sh, cat /proc/lab2_info, ls và cat ba thuộc tính /sys/class/lab2_class/lab2/, dmesg | grep lab2. Đối chiếu kết quả thực với D.5 trước khi thay placeholder.

• Nếu đã tạo theo bước C.3/C.4: nội dung info.txt, logs/boot.log trước và sau ghi nối tiếp; phục vụ log file mẫu Phần C.

• Kiểm tra tệp logs/boot_log.txt, source có comment theo 5.2 và gói MSSV_HoTen_Lab02.tar.gz chứa PDF, initramfs, module và script trước khi nộp.

13. Trình tự bổ sung minh chứng

Các lệnh dưới đây là kế hoạch thu bằng chứng, chưa phải kết quả đã chạy. Chỉ thay placeholder sau khi đã đối chiếu ảnh hoặc tệp log thực tế.

Bước 1. Kiểm tra toolchain và module trên Ubuntu Host

Chạy trong Terminal Ubuntu Host, tại thư mục LAB02:

```
cd "$HOME/LOS201/lab02"
arm-linux-gnueabihf-gcc --version | head -n 1
file driver/lab2_driver.ko
modinfo driver/lab2_driver.ko
```

Cần thấy tên/phiên bản compiler, file ELF ARM 32-bit và các trường của modinfo (đặc biệt vermagic tương thích kernel đích 5.15.0). Chụp trọn các trường; nếu output dài, dùng hai ảnh liên tiếp. Không xem modinfo trên Host là log nạp module trong QEMU.

Bước 2. Boot Phần D và thu log trong QEMU ARM

Trên Ubuntu Host, tạo thư mục logs và có thể dùng script để ghi lại phiên Terminal trước khi chạy lệnh QEMU của mục D.4:

```
mkdir -p "$HOME/LOS201/lab02/logs"
script -q -f "$HOME/LOS201/lab02/logs/boot_log.txt"
```

Trong shell do script mở, chạy đúng lệnh qemu-system-arm ở D.4. Sau khi vào dấu nhắc / #, các lệnh sau chạy bên trong QEMU ARM:

```
lsmod
ls -l /dev/lab2
test_driver.sh
test_procfs.sh
cat /proc/lab2_info
ls /sys/class/lab2_class/lab2/
for name in buffer_len open_count last_data; do
    printf "%s: " "$name"
    cat "/sys/class/lab2_class/lab2/$name"
done
dmesg | grep lab2
```

Kết quả cần kiểm tra: lsmod có lab2_driver; /dev/lab2 là character node major 240, minor 0; hai script kết thúc thành công; /proc và ba thuộc tính sysfs đọc được và phản ánh các lần ghi/mở; dmesg có log của driver. Số đếm và dữ liệu cuối phụ thuộc thứ tự các script, nên ghi đúng kết quả thực tế, không dùng lại số ở ảnh Phần B. Chụp riêng màn hình kết quả hai script và phần /proc, /sys, dmesg. Khi xong, thoát QEMU theo cách đã dùng trong buổi thực hành, rồi gõ exit tại shell Host do script mở để đóng boot_log.txt.

Bước 3. Hoàn thiện Phần C và hồ sơ nộp

Nếu cần bổ sung info.txt và logs/boot.log, trước mọi thao tác MTD phải kiểm tra cat /proc/mtd và tên của /sys/class/mtd/mtd1/name. Trên máy thực hành, mtd0 là BIOS và mtd1 từng là NAND simulator; không chạy flash_erase/nandwrite theo số thứ tự cũ khi chưa xác minh thiết bị hiện tại. Sau khi có các ảnh C còn thiếu, kiểm tra comment trong lab2_driver.c theo mục 5.2, tệp logs/boot_log.txt, các script, module ARM và initramfs; cập nhật PDF rồi mới đóng gói MSSV_HoTen_Lab02.tar.gz theo mục 5.1 của đề.